

Código diagrama clase Python



Generemos el código del diagrama de clase en Python

El diagrama UML representa una clase muy sencilla llamada **Automovil**, con:

- Un atributo privado: `velocidad`
- Dos métodos públicos:
 - `acelerar()`
 - `frenar()`

En Python no existe el modificador `private` como en Java o C#, pero por convención se utiliza un guion bajo (`_`) para indicar que un atributo es de uso interno.

Código en Python

</> Python

```
class Automovil:
    def __init__(self):
        self._velocidad = 0

    def acelerar(self):
        self._velocidad += 10
        print(f"El automóvil acelera. Velocidad actual: {self._velocidad} km/h")

    def frenar(self):
        if self._velocidad >= 10:
            self._velocidad -= 10
        else:
            self._velocidad = 0
```

```
print(f"El automóvil frena. Velocidad actual: {self._velocidad} km/h")
```

Programa principal para probar la clase

</> Python

```
# Crear un objeto de la clase Automovil
auto = Automovil()

# Acelerar dos veces
auto.acelerar()
auto.acelerar()

# Frenar tres veces
auto.frenar()
auto.frenar()
auto.frenar()
```

Salida esperada

```
El automóvil acelera. Velocidad actual: 10 km/h
El automóvil acelera. Velocidad actual: 20 km/h
El automóvil frena. Velocidad actual: 10 km/h
El automóvil frena. Velocidad actual: 0 km/h
El automóvil frena. Velocidad actual: 0 km/h
```

Explicación del código

- `class Automovil:` define la clase.
- `__init__()` es el **constructor**, que inicializa la velocidad en **0 km/h**.
- `_velocidad` representa el atributo privado indicado por el signo - en el diagrama UML.
- `acelerar()` incrementa la velocidad en 10 km/h.
- `frenar()` disminuye la velocidad en 10 km/h sin permitir que sea negativa.

Esta implementación respeta fielmente el diagrama UML mostrado y es un buen ejemplo introductorio de Programación Orientada a Objetos en Python.

